

Special Online Edition for UVA Online Judge Users

Art of Programming Contest

C Programming | Data Structures | Algorithms

2nd Edition

AHMED SHAMSUL AREFIN

FOREWORD BY

MIGUEL A. REVILLA
UNIVERSITY OF VALLADOLID, SPAIN

REVIEWED BY

STEVEN HALIM
DR. M. LUTFAR RAHMAN

ACMSOLVER Training Series



An Essential Book for Programmers

ART DEPAGPROGRAMACIÓNdoCONTEST

Tutoriales de programación en C | Estructuras de datos | Algoritmos

Compilado por

Ahmed Shamsul Arefin

Estudiante de posgrado,
Instituto de Tecnologías de la Información y la Comunicación de la
Universidad de Ingeniería y Tecnología de Bangladesh (BUET)
Licenciatura en Ciencias de la Computación e Ingeniería, CUET

Revisado por

Steven Halim

Facultad de Informática, Universidad Nacional de Singapur
Singapur.

Dr. M. Lutfar Rahman

Profesor del Departamento de Ciencias de la Computación e Ingeniería
Universidad de Dhaka.

Prólogo de

Profesor Miguel A. Revilla

Miembro del Comité Directivo Internacional de ACM-ICPC y Archivista de Problemas
Universidad de Valladolid,
España.

<http://acmicpc-live-archive.uva.es>

<http://online-judge.uva.es>



Gyankosh Prokashoni, Bangladesh

ISBN 984-32-3382-4

DDEDICADO A

Shahriar Manzoor

Juez de las Finales Mundiales ACM/ICPC 2003-2006

(Cuyos correos, publicaciones y problemas son de incalculable valor para todos los programadores)

Y

Mis queridos padres y colegas

AAGRADECIMIENTOS

Quisiera agradecer a las siguientes personas por su apoyo y ayuda para la mejora significativa de mis modestos trabajos. De hecho, esta lista aún está incompleta.

Profesor Miguel A. Revilla
Dr. M Kaykobad
Dr. M. Zafar Iqbal

Universidad de Valladolid, España.
Universidad Norte-Sur, Bangladesh.
Universidad de Ciencia y Tecnología
Shahjalal, Bangladesh.

Dr. M. Lutfar Rahman
Dr. Abu Taher
Howard Cheng
Steven Halim

Universidad de Dhaka, Bangladesh
Universidad Internacional Daffodil
Universidad de Lethbridge, Canadá
Universidad Nacional de Singapur,
Singapur

Shahriar Manzoor
Carlos Marcelino Casas Cuadrado
Mahbub Murshed Suman
Salahuddin Mohammad Masum
Samiran Mahmud

Universidad del Sudeste Asiático,
Bangladesh; Universidad de Valladolid,
España; Universidad Estatal de Arizona, EE.
UU.; Universidad Internacional Daffodil;
Universidad de Ingeniería y Tecnología de
Dhaka.

MH Rasel

Universidad de Ingeniería y Tecnología de
Chittagong

Sadiq M. Alam

Universidad Nacional de Singapur,
Singapur

Mehedi Bakht

Universidad de Ingeniería y Tecnología de
Bangladesh

Ahsan Raja Chowdhury
Mohammad Rubaiyat Ferdous Joya
KM Hasan
Monirul Islam Sharif
Gahangir Hossain

Universidad de Dhaka
Universidad de Toronto, Canadá
Universidad Norte-Sur
Instituto Tecnológico de Georgia, EE. UU.
Universidad de Ingeniería y Tecnología de
Chittagong

SM Saif Shams

Universidad de Ciencia y Tecnología de
Shahjalal

Shah Md. Shamsul Alam

Universidad Internacional Daffodil



Biografía del autor: Ahmed Shamsul Arefin está completando su maestría en **Universidad de Ingeniería y Tecnología de Bangladesh (BUET)** y ha completado una licenciatura en Ciencias de la Computación e Ingeniería de **CUET** en Ciencias de la Computación e Ingeniería. Participó en el Concurso Regional ACM 2001 en Dhaka, y su equipo quedó clasificado en el puesto 10. Se convirtió en organizador del concurso en **Juez en línea de Valladolid** organizando el "Concurso de Programación de Rockford 2001" y concursos locales en varias universidades. Su sitio web de capacitación para concursos de programación. "**ACMSolver.org**" Se ha vinculado con ACM UVA, USU y Polish Online Judge – Sphere.

Sus intereses de investigación son **Concursos, algoritmos, teoría de grafos y Aplicaciones basadas en la web**. Su correo electrónico de contacto: asarefin@yahoo.com Web: <http://www.daffodilvarsity.edu.bd/acmsolver/asarefin/>

Prefacio a 2^a Edición

Me complace poder presentar la 2ª edición de este libro a los lectores. El objetivo de esta edición no es solo ayudar a los concursantes durante las horas del concurso, sino también describir los temas centrales de **Ciencias de la Computación** como **Programación en C, Estructuras de datos y Algoritmos**. Esta edición supone una mejora respecto a la edición anterior. Incluye algunas técnicas de programación adicionales, como... **STL (Biblioteca de plantillas estándar)**. Aquí se introducen la manipulación de cadenas y el manejo de funciones matemáticas.

Se espera que la nueva edición sea bien recibida por todos aquellos a quienes va dirigida y que se convierta en un **Libro esencial para estudiantes de Informática**.

Prefacio al 1^a Edición

¿Por qué les encantan los concursos de programación a los programadores? Porque a los jóvenes programadores les gusta competir por la fama y el dinero, y les apasionan los algoritmos. El primer concurso regional asiático ACM-ICPC (International Collegiate Programming Contest) de Bangladesh se celebró en la Universidad North South en 1997. Excepto en el año 2000, nuestro país ha sido sede de este concurso anualmente, y nuestros valiosos programadores han participado en la final mundial cada año desde 1997.

Nuestro desempeño en ACM/ICPC mejora día a día. Se nota la atención y el tiempo que dedicamos a resolver problemas de dificultad moderada. BUET, la Universidad de Dhaka, NSU y AIUB han formado a muchos programadores que han competido en las Finales Mundiales. Las instituciones que buscan mejorar el desempeño de sus equipos en concursos de programación pueden considerarlas como posibles entrenadores. Algunas universidades han adoptado recientemente otra estrategia: ofrecen cursos de un crédito para estudiantes interesados en perfeccionar sus habilidades de programación y resolución de problemas.

Estoy muy agradecido a nuestros mentores, **Dr. M. Kaykobad** quien fue galardonado con el premio al "Mejor Entrenador" en las Finales Mundiales en Honolulu. Bajo su dinámica dirección, nuestros equipos nacionales se coronaron campeones en varias ocasiones en el Campeonato Regional ACM/ICPC de Asia. **Dr. M. Zafar Iqbal**, Juez principal de nuestros concursos regionales ACM/ICPC. **Dr. Abul L. Haque**, quien contactó primero **Dr. CJ Hwang** (Director de concursos de Asia y profesor en la Universidad Estatal de Texas, San Marcos, EE. UU.) y quería tener una sede regional de ACM/ICPC en Dhaka allá por 1997. También se debe agradecer enormemente a **Señor Shahriar Manzoor** Nuestro reconocido creador de problemas, director de jueces para la final regional (sede de Dhaka) y mundial de ACM/ICPC, y creador de problemas. Me gustaría agradecerle personalmente porque me mostró el camino correcto en varias ocasiones cuando creaba problemas para el juez en línea de Valladolid en el "Concurso de Programación de Rockford 2001" y mientras desarrollaba mi sitio web de capacitación para concursos de programación "ACMSolver.org".

Gracias a **Profesor Miguel A. Revilla**, Universidad de Valladolid, España por enlazar mi ACMSolver (<http://www.acmsolver.org>) sitio con su mundialmente famoso Juez Online de Valladolid (<http://acm.uva.es/p>) y nombrándome miembro del equipo algorítmico de jueces en línea de ACM Valladolid por ayudarlos a agregar algunos problemas en el archivo en vivo.

Y también un agradecimiento invaluable a **Steven Halim**, un estudiante de doctorado de la NUS, Singapur para el permiso de usar su sitio web (<http://www.comp.nus.edu.sg/~stevenha/>) contenido. Gran parte de este libro está compilado a partir de su renombrado sitio web. Por supuesto, cabe mencionar que su sitio web se basa en la página de entrenamiento de USACO ubicada en (<http://ace.delos.com/>)

Estoy agradecido **Universidad Internacional Daffodil**, especialmente al honorable Vicerrector **Profesor Aminul Islamy** Decano de la Facultad de Ciencias y Tecnologías de la Información **Dr. M. Lutfar Rahmany** todos mis colegas en **Departamento de Ciencias de la Computación e Ingeniería** Estoy aquí, por brindarme la oportunidad de oro de hacer algo en el Concurso de Programación de ACM y otras investigaciones.

Además, dado que este proyecto es una colección de tutoriales de varias fuentes, se reconoce a todos los autores de los tutoriales en el **Referencia** Sección de este libro. Localizar a los autores originales de algunos de estos tutoriales es bastante difícil. He intentado identificarlos caso por caso y en cada uno he solicitado permiso. Pido disculpas de antemano si hay algún error. Si es así, por favor, avísenme para que pueda mencionar el nombre en futuras ediciones.

Finalmente, quisiera añadir una frase al final de este prefacio. Durante los últimos años, mientras creaba y mantenía mi sitio web sobre el Concurso de Programación de la ACM, adquirí cierta experiencia. Sentí que era necesario ofrecer una guía para principiantes que quisieran adentrarse en el mundo de la programación. Por ello, comencé a recopilar tutoriales y a publicarlos en mi sitio web. Además, este es otro intento de promover el Concurso de Programación en nuestro país, ya que he procurado plasmar toda mi colección en formato impreso. Sus sugerencias serán bienvenidas.

Atentamente,

Ahmed Shamsul Arefin.



UNIVERSIDAD DE
VALLADOLID

Nota del prólogo

Como principal responsable del Online Judge de la Universidad de Valladolid, tengo la sensación de que este libro no es solo una recopilación de tutoriales, como afirma el autor en el prefacio, sino que también se convertirá en una parte esencial de las secciones de ayuda del sitio web de la UVa, ya que reúne mucha información dispersa sobre el Online Judge que puede ser de gran ayuda para muchos programadores de todo el mundo, sobre todo para los principiantes, algo fundamental para nosotros. El autor demuestra un especial interés en guiar al lector, y sus consejos deben considerarse casi como instrucciones, pues son fruto de una amplia experiencia tanto en la resolución de problemas como en la creación de los mismos. Por supuesto, el libro es mucho más que un manual de usuario del Online Judge y contiene información muy importante que falta en nuestra web, como la interesante clasificación de muchos problemas por categorías, que se analizan en detalle y con ejemplos. Creo que es un libro al que todos nuestros usuarios deberían tener acceso, ya que es un complemento perfecto para nuestro Online Judge.

Miguel A. Revilla

**Miembro del Comité Directivo Internacional de ACM-ICPC y Archivero de Problemas de la
Universidad de Valladolid, España.**

<http://acmicpc-live-archive.uva.es>

<http://online-judge.uva.es>

University Of Dhaka



Nota de revisión

Un concurso de programación informática es un evento placentero para los programadores principiantes, pero existen pocos libros disponibles como manuales de entrenamiento para competiciones de programación.

Este libro está diseñado para servir como texto principal en un curso de algoritmos enfocado en programación, así como en un curso de programación enfocado en algoritmos. El libro está especialmente diseñado para preparar a los estudiantes para participar en concursos como el ACM International Collegiate Programming Contest.

El libro abarca varios temas importantes relacionados con el desarrollo de habilidades de programación, tales como: conceptos fundamentales de concursos, plan de juego para un concurso, estructuras de datos esenciales para concursos, técnicas de entrada/salida, método de fuerza bruta, matemáticas, ordenación, búsqueda, algoritmos voraces, programación dinámica, grafos, geometría computacional, categoría de problemas del Concurso en línea de Valladolid, problemas de programación ACM seleccionados, códigos/rutinas comunes para programación, Biblioteca de plantillas estándar (STL), PC-Administración del concurso y guía para equipos. El libro también incluye una lista de sitios web y libros importantes para programadores de ACM/ICPC.

Creo que este libro será de gran utilidad para los jóvenes programadores interesados en participar en concursos de programación.

A handwritten signature in black ink, appearing to read 'M. Lutfar Rahman'.

Dr. M. Lutfar Rahman

Profesor del Departamento de Ciencias de la Computación e Ingeniería (CSE)

Universidad de Dhaka.

Bangladesh.



Notas de Steven Halim

Cuando ^I creado ^{mi} propio ^{sitio web} Mundo ^{de Siete} ^{pocos} años atrás (<http://www.comp.nus.edu.sg/~stevenha>) Mi objetivo era promover la comprensión de las estructuras de datos y los algoritmos, especialmente en el contexto de los concursos de programación, y motivar a más programadores a ser más competitivos ofreciendo numerosas pistas para los problemas del concurso en línea de la Universidad de Valladolid (UVa). Sin embargo, debido a mi apretada agenda, nunca logré dedicar tiempo a publicar adecuadamente el contenido de mi sitio web en formato de libro. Por ello, me alegra que Ahmed haya recopilado este libro y que haya obtenido mi autorización para hacerlo. Espero que este libro sea útil para los programadores en general, pero sobre todo para los programadores de Bangladesh, donde se venderá.

A handwritten signature in black ink, appearing to be 'SH' followed by a horizontal line.

Steven Halim
Universidad Nacional de Singapur (NUS),
Singapur.

Contenido

Capítulo 1	Conceptos fundamentales	14
Capítulo 2	Plan de juego para un concurso de programación	19
Capítulo 3	en C: un tutorial Estructuras de datos esenciales	27
Capítulo 4	para técnicas de entrada/salida en concursos	72
Capítulo 5		81
Capítulo 6	Método de fuerza bruta	85
Capítulo 7	Matemáticas	91
Capítulo 8	Clasificación	106
Capítulo 9	Búsqueda	113
Capítulo 10	Algoritmos voraces	117
Capítulo 11	Programación dinámica	121
Capítulo 12	Gráficos	134
Capítulo 13	Geometría Computacional	172
Capítulo 14	Valladolid OJ Categoría de problema Problemas de	174
Apéndice A	programación ACM Códigos/rutinas comunes para	176
Apéndice B	la programación Biblioteca de plantillas estándar	188
Apéndice C	(STL)	230
Apéndice D	ordenador personal: Administración de concursos y guía para	235
Apéndice E	equipos. Sitios web/libros importantes para ACM.	242
	Programadores	



acm International Collegiate
Programming Contest

IBM | event
sponsor

¿Qué es el concurso de programación de la ACM?

La Asociación para la Maquinaria de Computación (ACM) patrocina un concurso anual de programación, recientemente con el patrocinio de IBM. El concurso es muy conocido y muy respetado: el año pasado compitieron 2400 equipos de más de 100 naciones en los niveles regionales. Sesenta de ellos pasaron a las finales internacionales. Este concurso es conocido como **Concurso Internacional Universitario de Programación de la ACM (ICPC)**.

El concurso regional se celebra normalmente en noviembre, y la final en marzo. Equipos de tres estudiantes utilizan C, C++ o Java para resolver entre seis y ocho problemas en cinco horas. Cada equipo dispone de un ordenador, lo que permite a uno o dos miembros dedicarse a desarrollar una estrategia. A menudo, decidir qué problemas abordar primero es la habilidad más importante del concurso. Los problemas ponen a prueba tanto la identificación de los algoritmos subyacentes como la destreza y la velocidad de programación.

CAPÍTULO 1 CONCEPTOS FUNDAMENTALES

El concurso de programación es un excelente espacio para que los programadores exploren su potencial. Para empezar a resolver problemas, primero debes definir tu objetivo. Algunos participantes buscan aumentar la cantidad de problemas resueltos, mientras que otros prefieren resolver menos problemas, pero con mayor eficiencia. Elige una de estas dos categorías y comienza. Un participante sin un objetivo claro jamás podrá tener éxito en concursos online de 24 horas. Así que, piensa bien en tu objetivo.^[1]

Si eres principiante, intenta primero encontrar los problemas más fáciles. Intenta resolverlos en poco tiempo. Al principio, puede que necesites cada vez más tiempo para resolver incluso los problemas más sencillos. Pero no te desanimes. Se debe a tu falta de práctica. Intenta resolver problemas más fáciles, ya que aumentarán tu habilidad para programar. Muchos principiantes dedican mucho tiempo a programar en un lenguaje específico, pero para ser un buen programador no deberías dedicar tanto tiempo a programar, sino más bien a depurar y pensar en el algoritmo para cada problema. Un buen programador dedica el 10% de su tiempo a programar, el 45% a pensar y el resto a depurar. Por lo tanto, para reducir el tiempo de programación, debes practicar resolviendo primero los problemas más fáciles.

No intente usar un archivo de entrada para la entrada ni ningún archivo de salida para la salida cuando envíe el programa a los jueces en línea. Todas las partes de entrada y salida deben hacerse utilizando entradas y salidas estándar. Si es un programador de C o C++, pruebe esto: mientras codifica y depura errores, agregue las líneas en la primera línea del procedimiento principal, es decir:

```
#incluir <stdio.h>
principal ()
{
freopen("NOMBRE_DE_ARCHIVO_DE_ENTRADA","r",stdin);
freopen("NOMBRE_DE_ARCHIVO_DE_SALIDA","w",stdout); El
resto del código...
devolver 0;}
```

Pero al enviar a jueces en línea, elimine las dos líneas con `freopen` para evitar errores de función restringida. Si usa el primer `freopen` anterior, hará que su programa tome la entrada del archivo "FILE_NAME_FOR_INPUT". Escriba las entradas en el archivo para evitar ingresar la entrada varias veces para la depuración. Ahorra mucho tiempo. Pero como la función abre un archivo de entrada, lo que puede ser una causa de pirateo de los sitios web de los jueces en línea, no permiten usar la función y si la usa, darán un error de compilación (Función restringida). El segundo `freopen` es para generar la salida de su programa en un archivo específico llamado "FILE_NAME_FOR_OUTPUT" en la máquina. Es muy útil cuando la salida no se puede justificar solo viendo la ventana de salida (especialmente para el problema de manipulación de cadenas donde incluso un solo carácter de espacio puede ser la causa de

(Respuesta incorrecta). Para obtener más información sobre la función, consulte la Red de desarrolladores de Microsoft (Colección MSDN) y la ayuda sobre programación en C.

Lenguajes de programación y depuración sucia

La mayoría de las veces, un principiante se enfrenta al problema de decidir qué lenguaje de programación usar para resolver sus problemas. Por eso, a veces usa un lenguaje que no conoce a fondo. Depura durante horas buscando fallos y, finalmente, se da cuenta de que el problema no está en el algoritmo, sino en el código escrito en ese lenguaje en particular. Para evitar esto, intenta aprender un solo lenguaje de programación en profundidad y luego explora otros lenguajes más flexibles. Los lenguajes más comunes son C, C++, Pascal y Java. Java es el menos utilizado. Evita la depuración desordenada.

Evite errores de compilación

La respuesta más común al principiante del juez en línea de 24 horas es ERROR DE COMPILACIÓN (CE). Los consejos son:

- 1) Al usar una función, consulte la ayuda y verifique si está disponible en la forma estándar del lenguaje. Por ejemplo, no utilice la función ``strrev`` del archivo de cabecera ``string.h`` de C y C++, ya que no cumple con el estándar ANSI C ni C++. Si la necesita, deberá crearla manualmente. Codifique manualmente o evite aquellas funciones que estén disponibles en su compilador pero no en la forma estándar del lenguaje.
- 2) No uses archivos de entrada y salida para tu programa. Toma todas las entradas para la entrada estándar y escribe todas las salidas en la salida estándar (normalmente en la ventana de la consola). Consulta mis temas anteriores.
- 3) No utilice el archivo de cabecera `conio.h` en C o C++, ya que no está disponible en los estándares de C y C++. Generalmente, no utilice ninguna función disponible en el archivo de cabecera `<conio.h>`. Esto suele ser la causa principal de errores de compilación para los programadores que utilizan compiladores tipo Turbo C++.
- 4) No está permitido utilizar funciones y paquetes integrados en el juez en línea.
- 5) No envíes tu programa por correo electrónico, es decir, no uses Yahoo, Hotmail, etc., para enviarlo al juez, ya que es un método complejo: escribir el ID del juez, el número del problema, etc. En su lugar, usa el sistema de envío del juez en línea, por ejemplo, la página de envío de Valladolid. Usar el primero te dará CE la mayoría de las veces, ya que incluyen sus anuncios al principio y al final de tu programa. Así, el juez no puede reconocer los caracteres adicionales concatenados en tu código enviado y te da CE. Alrededor del 90% de los CE que hemos obtenido son por esto.

razón. El sistema de correo también divide sus programas en varias líneas, lo que provoca respuestas incorrectas u otros errores, aunque su programa fuera correcto.

Existen muchos jueces en línea de renombre que pueden evaluar tus códigos de solución las 24 horas. Algunos de ellos son:

Valladolid OJ (<http://acm.uva.es/p>)
Ural OJ (<http://acm.timus.ru>) Saratov
OJ (<http://acm.sgu.ru>) ZJU OJ (<http://acm.zju.edu.cn>)
Archivo oficial en directo de ACM (<http://cii-judge.baylor.edu/>)
Desafíos de programación del Juez en línea de la Universidad de Pekín (<http://acm.pku.edu.cn/JudgeOnline/>) (<http://www.programming-challenges.com>)

Olvidate de la eficiencia y empieza a resolver problemas más fáciles.

A veces, puede que notes que muchos programadores han resuelto muchos problemas pero han enviado muy pocos (¡son unos genios!). Al principio, podrías pensar que deberías intentar resolver los problemas con la menor cantidad de intentos posible. Así, después de resolver un problema, no querrás volver a intentarlo con otro algoritmo (que podría ser mucho mejor que el anterior) para actualizar tu posición en la clasificación. Pero mi opinión es que si piensas así, estás equivocado. Deberías probar otras maneras, ya que solo así podrás saber qué algoritmo es mejor. De nuevo, así podrás conocer los diversos errores que pueden ocurrir. Si no envías tu solución, no puedes saberlo. Quizás un problema que resolviste se pueda resolver en menos tiempo de otra manera. Por lo tanto, mi opinión es que pruebes todas las maneras que conozcas. En resumen, si eres principiante, olvídate de la eficiencia.

Busca los problemas más sencillos. Estos problemas se denominan problemas ad hoc. Puedes encontrar la lista de problemas disponibles en 24 OJ en los sitios web de S. Halim, acmbeginner y acmsolver. Intenta resolverlos y así podrás mejorar tus habilidades de programación.

Aprender algoritmos

La mayoría de los problemas de los Jueces en Línea dependen de varios algoritmos. Un algoritmo es una forma definida de resolver un problema en particular. Si ahora tienes habilidad para codificar y resolver problemas más fáciles, lee los libros de algoritmos a continuación. Por supuesto, debes tener muy buenas habilidades matemáticas para entender varios algoritmos. De lo contrario, no hay otra manera que saltarse los temas de los libros. Si tienes habilidad en matemáticas, lee los algoritmos uno por uno, intenta entenderlos. Después de entender los algoritmos, intenta escribirlos en el lenguaje de programación que has aprendido (esto se debe a que la mayoría de los

Los algoritmos se describen en pseudocódigo. Si puedes escribirlo sin errores, intenta encontrar problemas relacionados con el algoritmo y resolverlos. Existen muchos libros de algoritmos reconocidos. Intenta crear un algoritmo modificado a partir de los algoritmos presentados en el libro para resolver los problemas.

Utiliza algoritmos sencillos que garanticen la solución del problema, aunque no sean la solución óptima ni la más elegante. Úsalos incluso si son la solución más tonta, siempre que funcionen y no sean exponenciales. No compites por la elegancia o la eficiencia del algoritmo. Simplemente necesitas un algoritmo correcto, y lo necesitas ya. Cuanto más sencillo sea el algoritmo, mayores serán las probabilidades de que lo programes correctamente en tu primer intento.

Este es el consejo más importante a seguir en un concurso de programación. No tienes tiempo para diseñar algoritmos complejos una vez que tienes uno que cumple su función. Según el tamaño de tu entrada, puedes implementar incluso el algoritmo más simple y tener una solución funcional en un instante. No subestimes las CPU actuales. Un bucle for con 10 millones de repeticiones se ejecutará en un abrir y cerrar de ojos. E incluso si tarda 2 o 3 segundos, no te preocupes. Solo necesitas un programa que termine en un par de segundos. Normalmente, el tiempo límite para las soluciones se establece en 30 segundos o más. La experiencia demuestra que si tu algoritmo tarda más de 10 segundos en terminar, probablemente sea exponencial y deberías mejorarlo.

Obviamente, este consejo no debe seguirse al escribir código crítico que necesita estar lo más optimizado posible. Sin embargo, en mis pocos años de experiencia, solo nos hemos encontrado con este tipo de código crítico en controladores de dispositivos. Hablamos de rutinas que se ejecutan miles de veces por segundo. En estos casos, cada instrucción cuenta. De lo contrario, no vale la pena invertir 5 horas para una mejora del 50 % en una rutina que tarda 10 milisegundos en completarse y se llama cada vez que el usuario pulsa un botón. Nadie notará la diferencia. Solo tú lo sabrás.

Codificación sencilla

1. Evite el uso de los operadores ++ o -- dentro de expresiones o llamadas a funciones. Siempre úselos en una instrucción separada. Si hace esto, hay no hay posibilidad que introduzcas un error debido al incremento posterior o previo. Recuerda que esto no afecta al código de salida generado.
2. Evite expresiones de la forma *p++.
3. Evite la aritmética de punteros. En lugar de (p+5) use p[5].

4. Nunca programes de esta manera:

```
devolver (x*y)+Func(t)/(1-s);
```

pero como:

```
temp = func(t);  
RetVal = (x*y) + temp/(1-s); devolver  
RetVal;
```

De esta forma puedes comprobar con tu depurador cuál fue el valor de retorno de Func(t) y cuál será el código de retorno de tu función.

5. Evite usar el operador =. En lugar de:

```
devolver (((x*8-111)%7)>5) ? y : 8-x;
```

En su lugar, utilice:

```
Temp = ((x*8-111)%7);                      si (5<Temp) devuelve y; de lo contrario    devolver 8-x;
```

Si sigues esas reglas, eliminarás cualquier posibilidad de errores triviales, y si necesitas depurar el código, te resultará mucho más fácil hacerlo.

- NOMENCLATURA 1: No uses nombres cortos y similares para tus variables. Si tienes tres variables de puntero, no las llames p1, p2 y p3. Usa nombres descriptivos. Recuerda que tu tarea es escribir código que, al leerlo, indique claramente su función. Usar nombres como Index, RightMost y Retries es mucho mejor que i, rm y rt. El tiempo que pierdes escribiendo más no es nada comparado con la ventaja de tener un código que se explica por sí mismo.

- NOMBRAMIENTO 2: Utilice la nomenclatura húngara, pero hasta cierto punto. Incluso si se opone a ella (lo cual, le guste o no, es un signo de inmadurez), es deinmenso ayuda.

- NOMENCLATURA 3: No utilice nombres como {i,j,k} para las variables de control de bucle. Utilice {I,K,M}. Es muy fácil confundir aj con i al leer código o al copiar, pegar y modificar código, pero no hay posibilidad de confundir I con K o M.

Últimas palabras

La práctica hace al maestro. Así que intenta resolver cada vez más problemas. Un genio no se forma de la noche a la mañana. Algún día podrías estar entre los diez primeros de la clasificación. Así que consigue un ordenador, instala un lenguaje de programación y empieza a resolver problemas de inmediato.^[1]

CAPÍTULO 2 PLAN DE JUEGO PARA UNA COMPETICIÓN

Durante una competición en tiempo real, equipos formados por tres estudiantes y un ordenador deben resolver tantos problemas como sea posible en 5 horas. Gana el equipo con más problemas resueltos, donde "resuelto" significa producir las salidas correctas para un conjunto de entradas de prueba (secretas). Aunque las habilidades individuales de los miembros del equipo son importantes, en Para ser un equipo de primera es necesario aprovechar la sinergia dentro del equipo.[2]

Sin embargo, para aprovechar al máximo una estrategia, también es importante que tus habilidades individuales estén lo más perfeccionadas posible. No hace falta ser un genio, ya que la práctica te puede llevar muy lejos. En nuestra filosofía, hay tres factores cruciales para ser un buen equipo de programación:

Conocimiento de algoritmos estándar y capacidad para encontrar un algoritmo apropiado para cada problema del conjunto;
Capacidad para codificar un algoritmo en un programa funcional; y
tener una estrategia de cooperación con los compañeros de equipo.

¿Qué es un algoritmo?

"Un buen algoritmo es como un cuchillo afilado: hace exactamente lo que se supone que debe hacer con un mínimo esfuerzo. Usar el algoritmo equivocado para resolver un problema es como intentar cortar un filete con un destornillador: puede que al final obtengas un resultado digerible, pero gastarás mucho más esfuerzo del necesario, y el resultado será

Es poco probable que resulte estéticamente agradable.

Un algoritmo es una secuencia de instrucciones paso a paso que la computadora debe seguir.

Para poder competir en concursos de programación, es necesario conocer muchos algoritmos conocidos y tener la capacidad de identificar qué algoritmos son adecuados para un problema en particular (si el problema es sencillo), o qué combinaciones o variantes de algoritmos (si el problema es un poco más complejo).

Un algoritmo bueno y correcto según los jueces en concursos de programación:

1. Debe terminar

[De lo contrario: se mostrará el mensaje "Límite de tiempo/Límite de memoria/Límite de salida excedido"]

2. Cuando finalice, debe producir una salida correcta [de lo contrario: se dará la famosa respuesta de Respuesta Incorrecta].

3. Debe ser lo más eficiente posible [de lo contrario: se indicará que se ha excedido el límite de tiempo].

Capacidad para identificar rápidamente los tipos de problemas.

En todos los concursos de programación, solo hay tres tipos de problemas:

1. No había visto este antes.
2. Ya he visto este tipo antes, pero no lo he resuelto o no puedo resolverlo.
3. Ya he resuelto este tipo de problemas antes.

En los concursos de programación, tendrás que lidiar con un conjunto de problemas. No se trata de un solo problema. La capacidad de identificar rápidamente los problemas en las clasificaciones de concursos mencionadas anteriormente (no los he visto, los he visto, los he resuelto) será un factor clave para tener éxito en los concursos de programación.

Matemáticas	Número primo
	Entero grande
	Permutación
	Teoría de números
	Factorial
	Fibonacci
	Secuencias
	Módulo
Programación dinámica	Subsecuencia común más larga
	Subsecuencia creciente más larga
	Editar distancia
	0/1 Mochila
	Cambio de monedas
	Multipliación de cadenas de matrices
	Suma máxima del intervalo
Gráfico	Recorrido
	Relleno de inundación
	Floyd Warshal
	MST
	Max Bipertite Matching
	Flujo de red
	Punto de articulación
Clasificación	Ordenación de burbuja
	Ordenación rápida
	Ordenación por fusión (DAndC)
	Ordenación por selección
	Ordenación por radix
	Clasificación por cubetas
Búsqueda	Búsqueda completa, fuerza bruta
	Búsqueda binaria (DAndC)
	BST
Simulación	Josefo
Procesamiento de cadenas	Coincidencia de cadenas
	Coincidencia de patrones
Geometría Computacional	Envolverte convexa
Ad Hoc	Problemas triviales

En ocasiones, el algoritmo puede estar "anidado" dentro de un bucle de otro algoritmo. Por ejemplo, una búsqueda binaria dentro de un algoritmo de programación dinámica, lo que dificulta la identificación del tipo de problema.

Si quieres poder competir bien en concursos de programación, debes saber todo lo que hemos mencionado anteriormente, teniendo en cuenta algunas precauciones ante problemas puntuales.

Los problemas «ad hoc» son aquellos cuyos algoritmos no se ajustan a las categorías estándar con soluciones bien estudiadas. Cada problema ad hoc es diferente... No existen técnicas específicas ni generales para resolverlos. Esto los convierte en problemas interesantes (y a veces frustrantes), ya que cada uno presenta un nuevo desafío.

Las soluciones podrían requerir una estructura de datos novedosa o un conjunto inusual de bucles o condicionales. A veces, requieren combinaciones especiales poco comunes o, al menos, raramente frecuentes. Generalmente, exigen una lectura atenta de la descripción del problema y suelen dar lugar a un ataque que se basa en la secuenciación cuidadosa de las instrucciones proporcionadas. Los problemas ad hoc aún pueden requerir optimizaciones razonables y, al menos, un cierto grado de análisis que permita evitar, por ejemplo, bucles anidados hasta cinco niveles.

Capacidad para analizar tu algoritmo

Has identificado tu problema. Crees saber cómo resolverlo. La pregunta que debes hacerte ahora es sencilla: dado el límite máximo de entrada (generalmente especificado en la descripción del problema), ¿puede mi algoritmo, con la complejidad que puedo calcular, superar el límite de tiempo establecido en el concurso de programación?

Por lo general, existen varias maneras de resolver un problema. Sin embargo, algunas pueden ser incorrectas y otras no son lo suficientemente rápidas. No obstante, la regla general es: **¡Piensa en muchos algoritmos posibles y luego elige el más tonto que funcione!**

Aspectos que aprender sobre algoritmos

1. Prueba de la corrección del algoritmo (especialmente para algoritmos voraces)
2. Análisis de complejidad temporal/espacial para algoritmos no recursivos.
3. Para algoritmos recursivos, el conocimiento del cálculo de relaciones de recurrencia y su análisis: método iterativo, método de sustitución, método del árbol de recursión y, finalmente, el Teorema Maestro.
4. Análisis de algoritmos aleatorios que implican conocimiento probabilístico, por ejemplo: variable aleatoria, esperanza matemática, etc.
5. Análisis amortizado.
6. Análisis sensible a la salida, para analizar algoritmos que dependen del tamaño de la salida, por ejemplo: algoritmo LIS $O(n \log k)$, que depende de k , que es el tamaño de la salida, no el tamaño de la entrada.

Tabla 1: Comparación temporal de diferentes órdenes de crecimiento

Suponemos que nuestro ordenador puede calcular 1000 elementos en 1 segundo (1000 ms).

Orden de crecimiento	norte	Tiempo (ms)	Comentario
$O(1)$	1000	1	Excelente, casi imposible en la mayoría de los casos.
$O(\log n)$	1000	9,96	Muy bien, ejemplo: Búsqueda binaria
$O(n)$	1000	1000	Algoritmo de tiempo lineal normal
$O(n \log n)$	1000	9960	En promedio, esto se suele encontrar en algoritmos de ordenación como la ordenación rápida.
$O(n^2)$	1000	1.000.000	Lento
$O(n^3)$	1000	10^9	Por cierto, el algoritmo de rutas más cortas entre todos los pares de elementos, Floyd-Warshall, es lento y tiene una complejidad de $O(N^3)$.
$O(2^n)$	1000	2^{1000}	Crecimiento exponencial deficiente... intente evitarlo. Use Dynamic Programming, si puedes.
$O(n!)$	1000	incontable	Problemas típicos NP-completos.

Tabla 2: Límite del tamaño máximo de entrada bajo el límite de tiempo de 60 segundos (con suposiciones)

Orden de crecimiento	Tiempo (ms)	Máximo posible n	Comentario
$O(1)$	60.000 ms	Prácticamente infinito	Mejor
$O(\log n)$	60.000 ms	$6^{18.000}$	Un número muy, muy grande
$O(n)$	60.000 ms	60.000	Sigue siendo un número muy grande
$O(n \log n)$	60.000 ms	~ 5.000	Tamaño moderado, tamaño promedio en la vida real
$O(n^2)$	60.000 ms	244	pequeño
$O(n^3)$	60.000 ms	39	pequeñito
$O(2^n)$	60.000 ms	16	evítalo si puedes.
$O(n!)$	60.000 ms	8	extremadamente demasiado pequeño.

Puede resultar útil memorizar el siguiente orden para determinar rápidamente qué algoritmo funciona mejor asintóticamente: **constante < log n < n < n log n < n^2 < n^3 < 2^n < n!**

Algunas reglas generales

1. La estructura de datos integrada más grande, "long long", es $2^{63}-1$: $9 \cdot 10^{18}$ (hasta 18 dígitos).
2. Si tienes k bucles anidados que se ejecutan aproximadamente n iteraciones cada uno, el programa tiene una complejidad de $O(n^k)$.
3. Si su programa es recursivo con b llamadas recursivas por nivel y tiene l niveles, la complejidad del programa es $O(b^l)$.
4. Ten en cuenta que hay n! permutaciones y 2^n subconjuntos o combinaciones de n elementos cuando se trabaja con este tipo de algoritmos.

5. Los mejores tiempos para ordenar n elementos son $O(n \log n)$.
6. Algoritmos de programación dinámica que implican rellenar una matriz, generalmente en $O(n^3)$.
7. En concursos, la mayoría de las veces los algoritmos $O(n \log n)$ serán suficientes.

El arte de probar tu código

Lo has conseguido. Has identificado el problema, diseñado el mejor algoritmo posible, calculado la complejidad temporal y espacial para que se ajuste a los límites de tiempo y memoria establecidos, y lo has programado a la perfección. Pero, ¿es 100% correcto?

Dependiendo del tipo de concurso de programación, es posible que obtengas crédito o no por resolver el problema parcialmente. En ACM ICPC, solo obtendrás crédito si el código de tu equipo resuelve todos los casos de prueba; eso es todo, recibirás "Aceptado" o "No aceptado" (Respuesta incorrecta, límite de tiempo excedido, etc.). En IOI, existe un sistema de crédito parcial, en el que obtendrás una puntuación: número de respuestas correctas dividido por el número total de casos de prueba para cada código que envíes.

En cualquier caso, deberá diseñar casos de prueba complejos, bien fundamentados y con un enfoque adecuado. El ejemplo de entrada-salida proporcionado en la descripción del problema es, por defecto, demasiado trivial y, por lo tanto, no es una buena manera de medir la corrección de su código para todas las instancias de entrada.

En lugar de desperdiciar tu envío (y obtener una penalización de tiempo o puntos) por dar una respuesta incorrecta, quizás quieras diseñar primero algunos casos de prueba complicados, probarlos en tu propia máquina y asegurarte de que tu código pueda resolverlos correctamente (de lo contrario, no tiene sentido enviar tu solución, ¿verdad?).

Algunos entrenadores de equipos a veces les piden a sus alumnos que compitan entre sí diseñando casos de prueba. Si los casos de prueba del alumno A logran romper el código de otro alumno, A obtendrá un punto extra. También puedes probar esto en el entrenamiento de tu equipo escolar.

Aquí tienes algunas pautas para diseñar buenos casos de prueba:

1. Debe incluir una entrada de ejemplo, la más trivial, incluso la respuesta proporcionada.
2. Debe incluir casos límite, cuál es el máximo de n , x , y u otras variables de entrada, intente variar sus valores para probar errores fuera de rango.
3. Para casos de prueba con múltiples entradas, intente usar dos casos de prueba idénticos de forma consecutiva. Ambos deben arrojar el mismo resultado. Esto sirve para comprobar si olvidó inicializar algunas variables, lo cual se identificará fácilmente si la primera instancia produce el resultado correcto pero la segunda no.
4. Aumenta el tamaño de la entrada. A veces, tu programa funciona correctamente con entradas pequeñas, pero se comporta de forma incorrecta cuando el tamaño de la entrada aumenta.
5. Casos de prueba complicados: analiza la descripción del problema e identifica las partes que son complicadas; pruébalas con tu código.
6. No asuma que la entrada siempre estará bien formateada si la descripción del problema no lo indica.

Entonces, intenta insertar espacios en blanco (espacios, tabulaciones) en tu entrada y verifica si tu código puede leer los valores correctamente.

7. Por último, realiza pruebas con casos aleatorios, prueba con entradas aleatorias y comprueba que tu código sea correcto.

Generando soluciones ganadoras

Una buena forma de obtener una ventaja competitiva es elaborar un plan de juego para cada ronda del concurso. Esto te ayudará a planificar tus acciones, tanto para cuando las cosas salgan bien como para cuando salgan mal. De esta manera, podrás dedicar tu tiempo de reflexión a resolver problemas de programación en lugar de pensar en qué hacer a continuación... es como precalcular tus reacciones ante la mayoría de las situaciones.

Leer hastatodo Primero, resuelve los problemas; no intentes resolver uno directamente, ya que podrías pasar por alto alguno más fácil.

1. Ordena los problemas: primero el más corto, en términos de esfuerzo (del más corto al más largo: ya lo has hecho antes, fácil, desconocido, difícil).
2. Describe los algoritmos, su complejidad, los números, las estructuras de datos y los detalles complicados.
3. Piensa en otros posibles algoritmos (si los hay) y luego elige el más tonto que funcione.
4. ¡Haz los cálculos! (complejidad espacial y temporal, e introduce los valores reales esperados y del peor caso).
5. Codifícalo, por supuesto, lo más rápido posible, y debe ser correcto.
6. Intenta romper el algoritmo: usa casos de prueba especiales (¿degenerados?).

Codificar un problema

1. Solo empieza a programar después de haber finalizado tu algoritmo.
2. Crea datos de prueba para casos complicados.
3. Codifique la rutina de entrada y pruébela (escriba rutinas de salida adicionales para mostrar los datos).
4. Codifique la rutina de salida y pruébela.
5. Escribe las estructuras de datos necesarias.
6. Refinamiento por etapas: escriba comentarios que describan la lógica del programa.
7. Introduce el código y depura una sección a la vez.
8. Haz que funcione y verifica su correcto funcionamiento (utiliza casos de prueba sencillos).
9. Intenta romper el código: utiliza casos especiales para comprobar la corrección del código.

Estrategia de gestión del tiempo y escenarios de "control de daños"

Ten un plan sobre qué hacer cuando varias cosas (¡previsibles!) salgan mal; imagina los problemas que podrías tener y decide cómo quieres reaccionar. La pregunta central es:

¿Cuándo se debe dedicar más tiempo a depurar un programa y cuándo es mejor dar por perdida la inversión y seguir adelante? Considere estos aspectos:

1. ¿Cuánto tiempo llevas depurándolo?
2. ¿Qué tipo de error parece tener?
3. ¿Su algoritmo es incorrecto?
4. ¿Es necesario modificar sus estructuras de datos?
5. ¿Tienes alguna idea de qué está fallando?
6. Dedicar poco tiempo (20 minutos) a depurar es mejor que cambiar a otra cosa; pero podrías resolver otro problema desde cero en 45 minutos.
7. ¿Cuándo retomas un problema que habías abandonado previamente?
8. ¿Cuándo dedicas más tiempo a optimizar un programa y cuándo cambias de estrategia?
9. Piensa a partir de ahora: olvida el esfuerzo anterior, concéntrate en el futuro: ¿cómo puedes obtener la mayor cantidad de puntos en la próxima hora con lo que tienes?

Consejos y trucos para concursos

1. Usa la fuerza bruta cuando puedas. El algoritmo de fuerza bruta suele ser el más fácil de implementar.
2. KISS: ¡Lo simple es inteligente! (¡Mantenlo simple, estúpido! / Mantenlo corto y simple).
3. Sugerencia: concéntrate en los límites (especificados en el enunciado del problema).
4. Desperdicia memoria cuando te facilite la vida (intercambia espacio de memoria por velocidad).
5. No elimine la información de depuración adicional, coméntela.
6. Optimizar progresivamente y solo en la medida necesaria.
7. ¡Conserve todas las versiones que funcionen!
8. Código para depurar:
 - a. El espacio en blanco es bueno,
 - b. utilizar nombres de variables significativos,
 - c. No reutilice variables (aquí no estamos haciendo ingeniería de software).
 - d. refinamiento por etapas,
 - e. Comentar antes del código.
9. Evita las indicaciones si puedes.
10. Evita la memoria dinámica como la peste: asigna todo de forma estática. (sí, sí)
11. Intenta no usar números de coma flotante; si tienes que hacerlo, incluye tolerancias en todas partes (nunca compruebes la igualdad).
12. Comentarios sobre los comentarios:
 - a. No es prosa extensa, solo notas breves.
 - b. Explique la funcionalidad de alto nivel: `++i`; `/*` aumenta el valor de `i` en `*` es peor que inútil.
 - c. Explique los trucos de código.
 - d. Delimitar y documentar las secciones funcionales.

Lleva un registro de tu desempeño en cada competencia: éxitos, errores y qué podrías haber hecho mejor; ¡usa esta información para reescribir y mejorar tu estrategia!

Los jueces son malvados y quieren hacerte daño.

Los jueces no quieren incluir problemas fáciles en el concurso, porque ya han ideado demasiados problemas difíciles. Así que lo que hacemos es ocultar los problemas fáciles, con la esperanza de que caigan en la trampa y se enfoquen en los más difíciles. Si les pedimos que sumen dos números no negativos, encontrarán páginas de texto sobre la adición del cero al sistema numérico e imágenes en 3D para explicar el proceso de suma tal como se imaginaba en alguna isla desconocida.

Una vez que te hayamos asustado con los problemas fáciles, haremos que los difíciles parezcan fáciles. «Dados dos polígonos, halla el área de su intersección». Fácil, ¿verdad?

No siempre es obvio que un problema sea fácil, por lo que los equipos lo ignoran o recurren a enfoques demasiado complejos. Recuerda que hay docenas de otros equipos trabajando en los mismos problemas, y ellos te ayudarán a encontrar los más sencillos. **Si todo el mundo está resolviendo el problema G, tal vez deberías volver a analizarlo.**^[6]

CAPÍTULO 3 PROGRAMACIÓN EN C: UN TUTORIAL^[11]

C fue creado por **Dennis Ritchie** en **Laboratorios Bell Telephone** en 1972. Debido a que C es un lenguaje tan potente y flexible, su uso se extendió rápidamente más allá de Bell Labs. Programadores de todo el mundo comenzaron a usarlo para escribir todo tipo de programas. Sin embargo, pronto diferentes organizaciones comenzaron a utilizar sus propias versiones de C, y las sutiles diferencias entre las implementaciones comenzaron a causar dolores de cabeza a los programadores. En respuesta a este problema, el **Instituto Nacional Estadounidense de Estándares (ANSI)** En 1983 se formó un comité para establecer una definición estándar de C, que se conoció como ANSI Standard C. Con pocas excepciones, todos los compiladores de C modernos tienen la capacidad de cumplir con este estándar [11].

Un programa sencillo en C

Un programa AC consta de una o más funciones, similares a las funciones y subrutinas de un programa Fortran o a los procedimientos de PL/I, y quizás algunas definiciones de datos externos. principales una función de este tipo, y de hecho todos los programas C deben tener una principal. La ejecución del programa comienza en la primera instrucción de principal. principal Por lo general, invocará otras funciones para realizar su trabajo, algunas provenientes del mismo programa y otras de bibliotecas.

```
principal() {  
    printf("hola, mundo"); }
```

imprimirfes una función de biblioteca que formateará e imprimirá la salida en la terminal (a menos que se especifique otro destino). En este caso imprime

Hola Mundo

Un programa funcional en C; Variables; Tipos y declaraciones de tipos

Aquí hay un programa más extenso que suma tres números enteros e imprime su suma.

```
principal() {  
    enteros a, b, c, suma; a = 1; b  
    = 2; c = 3; suma = a + b + c;  
  
    printf("la suma es %d", suma);  
}
```

La aritmética y las sentencias de asignación son muy similares a las de Fortran (excepto por los puntos y comas) o PL/I. El formato de los programas en C es bastante libre. Podemos poner varias

Podemos colocar instrucciones en una línea si lo deseamos, o podemos dividir una instrucción en varias líneas si parece conveniente. La división puede ser entre cualquiera de los operadores o variables, pero *no* en medio de un nombre u operador. Por estilo, se recomienda usar espacios, tabulaciones y saltos de línea libremente para mejorar la legibilidad.

C tiene tipos fundamentales de variables:

Tipo de variable	Palabra clave	Rango de bytes requerido	
Personaje	carbonizarse	1	- 128 a 127
Entero	entero	2	- 32768 a 32767
Entero corto	corto	2	- 32768 a 32767
Entero largo	largo	4	- 2.147.483.648 a 2.147.438.647
carácter sin firmar	carácter sin signo	1	0 a 255
Entero sin signo	entero sin signo	2	0 a 65535
entero corto sin signo	unsigned short	2	0 a 65535
Entero largo sin signo	largo sin firmar	4	De 0 a 4.294.967.295
Precisión simple	flotar	4	1.2E-38 a
punto flotante			3.4E38
Doble precisión	doble	8	2.2E-308 a
punto flotante			1.8E308

También existen matrices y estructuras de estos tipos básicos, punteros a ellas y funciones que las devuelven, todo lo cual veremos en breve.

Todas las variables en un programa C deben ser declaradas, aunque esto a veces puede hacerse implícitamente por el contexto. Las declaraciones deben preceder a las instrucciones ejecutables. La declaración

```
entero a, b, c, suma;
```

Los nombres de las variables tienen entre uno y ocho caracteres, elegidos entre AZ, az, 0-9 y `_`, y comienzan con un carácter que no sea un dígito.

Constantes

Ya hemos visto constantes enteras decimales en el ejemplo anterior: 1, 2 y 3. Dado que C se usa a menudo para la programación de sistemas y la manipulación de bits, los números octales son una parte importante del lenguaje. En C, cualquier número que comience con 0 (¡cero!) es un entero octal (y por lo tanto no puede tener ningún 8 ni ningún 9). Por lo tanto, `0777` es una constante octal, con valor decimal 511.

Un «carácter» es un byte (un concepto inherentemente dependiente de la máquina). Generalmente se expresa como una constante de carácter, que es un carácter entre comillas simples. Sin embargo, puede ser cualquier cantidad que quepa en un byte, como en las banderas que se muestran a continuación:

```
char quest, newline, flags; quest = '?';
```

```
salto de línea = '\n';
```

```
banderas = 077;
```

La secuencia `\n` es la notación C para el carácter de nueva línea, que, al imprimirse, salta al inicio de la siguiente línea. Nótese que `\n` representa un solo carácter. Existen otros caracteres de escape similares a `\n` para representar caracteres difíciles de obtener o invisibles, como `\t` para la tabulación, `\b` para el retroceso, `\0` para el final del archivo y `\\` para la barra invertida.

Entrada/salida simple: `getchar()`, `putchar()`, `printf()`

obtenerchar y putchar Las funciones básicas de la biblioteca de E/S en C son `getchar`. Esta función obtiene un carácter de la entrada estándar (generalmente la terminal) cada vez que se llama y devuelve ese carácter como valor. Cuando llega al final del archivo que está leyendo, devuelve el carácter representado por `\0` (NUL en ASCII, que tiene valor cero). Veremos cómo usarla en breve.

```
principal( ) {  
    carácter c;  
    c = obtener carácter();  
    poner carácter(c);  
}
```

La función `putchar` envía un carácter a la salida estándar (normalmente la terminal) cada vez que se llama. Por lo tanto, el programa anterior lee un carácter y lo vuelve a escribir. En sí mismo, esto no es muy interesante, pero observe que si agregamos un bucle y una comprobación de fin de archivo, obtenemos un programa completo para copiar un archivo a otro.

`printf` es una función más compleja para producir una salida formateada.

```
printf ("hola, mundo\n");
```

es el uso más sencillo. Se imprime la cadena "hello, world\n".

Más complicado, si sumas 6,

```
printf("la suma es %d\n", suma);
```

huellas dactilares

```
La suma es 6
```

Dentro del primer argumento de printf, los caracteres "%d" indican que el siguiente argumento en la lista de argumentos se imprimirá como un número en base 10.

Otros comandos de formato útiles son "%c" para imprimir un solo carácter, "%s" para imprimir una cadena completa y "%o" para imprimir un número en octal en lugar de decimal (sin cero inicial). Por ejemplo,

```
n = 511;
printf("¿Cuál es el valor de %d en octal?", n); printf("%s! %d decimal es %o\n", "Correcto", n, n);
```

huellas dactilares

```
¿Cuál es el valor de 511 en octal? ¡Correcto! 511 en decimal es 777 en octal.
```

Si -operadores relacionales y sentencias compuestas

La instrucción básica de prueba condicional en C es la instrucción if:

```
c = obtener carácter();
si( c == '?' )
    printf("¿Por qué escribiste un signo de interrogación?\n");
```

La forma más simple de **si** es

```
si(expresión)declaración
```

La condición a probar es cualquier expresión encerrada entre paréntesis. **Le sigue un** declaración. La expresión se evalúa y, si su valor no es cero, la instrucción es Hay una ejecutado. opción **demás** cláusula, que se describirá próximamente.

La secuencia de caracteres "==" es uno de los operadores relacionales en C; aquí está el conjunto completo:

```
==    igual a (.EQ.(a los fortraners) no es
!=    igual a
>     más que
```

<	<i>menos que</i>
> =	<i>mayor o igual que menor o igual</i>
<=	<i>que</i>

El valor de `expresión`relaciónLa expresión es 1 si la relación es verdadera y 0 si es falsa. No olvides que la prueba de igualdad es `==`; un solo `=` provoca una asignación, no una prueba, y siempre conduce a un desastre.

Las pruebas se pueden combinar con los operadores `&&` (AND), `||` (OR) y `!` (NOT). Por ejemplo, podemos comprobar si un carácter es un espacio en blanco, una tabulación o un salto de línea con

```
si( c==' ' || c=='\t' || c=='\n' ) ...
```

C garantiza que `&&` y `||` se evalúen de izquierda a derecha; pronto veremos casos en los que esto importa.

Como ejemplo sencillo, supongamos que queremos asegurarnos de quea es más grande queb, como parte de una rutina de clasificación. El intercambio dea ybtoma tres instrucciones en C, agrupadas entre {}:

```
si (a < b) {
    t = a;
    a = b;
    b = t;
}
```

Como regla general en C, dondequiera que pueda usar una instrucción simple, puede usar cualquier instrucción compuesta, que es simplemente una serie de instrucciones simples o compuestas encerradas entre llaves {}. No hay punto y coma después de las llaves de una instrucción compuesta, pero es un punto y coma después de la última instrucción no compuesta dentro de las {}.

MientrasDeclaración; Asignación dentro de una expresión; Declaración nula

El mecanismo básico de bucle en C es la instrucción while. Aquí tienes un programa que copia su entrada a su salida carácter por carácter. Recuerda que `'\0'` marca el final del archivo.

```
principal( ) {
    carácter c;
    while( (c=getchar( )) != '\0' ) putchar(c);
}
```

La instrucción while es un bucle, cuya forma general es

<code>mientras (expresión) declaración</code>
--

Su significado es

(a) evaluar la expresión

(b) si su valor es verdadero (es decir, distinto de cero) realiza la instrucción y vuelve a (a).

Como la expresión se evalúa antes de ejecutar la instrucción, esta última puede ejecutarse cero veces, lo cual suele ser deseable. Al igual que en la instrucción `if`, tanto la expresión como la instrucción pueden ser arbitrariamente complejas, aunque aún no lo hemos visto. Nuestro ejemplo obtiene el carácter, lo asigna a `c` y luego comprueba si es un `'\\0'`. Si no lo es, se ejecuta la instrucción del bucle `while`, imprimiendo el carácter. El bucle `while` se repite. Cuando el carácter de entrada es finalmente un `'\\0'`, el bucle `while` termina, al igual que `main`.

Observe que utilizamos una instrucción de asignación.

<code>c = obtener_carácter()</code>

dentro de una expresión. Este es un atajo de notación muy útil que a menudo produce un código más claro. (De hecho, suele ser la única forma de escribir el código de forma limpia. Como ejercicio, reescribe la copia de archivos sin usar una asignación dentro de una expresión). Funciona porque una instrucción de asignación tiene un valor, al igual que cualquier otra expresión. Su valor es el valor del lado derecho. Esto también implica que podemos usar múltiples asignaciones como

<code>x = y = z = 0;</code>

La evaluación se realiza de derecha a izquierda.

Por cierto, los paréntesis adicionales en la instrucción de asignación dentro del condicional eran realmente necesarios: si hubiéramos dicho

<code>c = getchar() != '\\0'</code>

`c` se establecería en 0 o 1 dependiendo de si el carácter recuperado era el final del archivo o no. Esto se debe a que, en ausencia de paréntesis, el operador de asignación `'='` se evalúa después del operador relacional `'!='`. En caso de duda, o incluso si no la hay, use paréntesis.

<pre>principal() { mientras(putchar(getchar()) != '\\0'); }</pre>

¿Qué instrucción se repite? Ninguna, o técnicamente, la instrucción nula, porque todo el trabajo se realiza dentro de la parte de prueba del bucle while. Esta versión es ligeramente diferente de la anterior, porque el carácter '\0' final se copia a la salida antes de que decidamos detenernos.

Aritmética

Los operadores aritméticos son los habituales '+', '-', '*', y '/' (que trunca la división entera si ambos operandos son int), y el operador de resto o módulo '%':

```
x = a%b;
```

x se establece como el resto de la división de a entre b (es decir, a módulo b). Los resultados dependen de la máquina, a menos que tanto a como b sean positivos.

En aritmética, las variables de tipo char generalmente se pueden tratar como variables de tipo int. Las operaciones aritméticas con caracteres son perfectamente válidas y, a menudo, tienen sentido.

```
c = c + 'A' - 'a';
```

Convierte un único carácter ASCII en minúscula almacenado en C a mayúscula, aprovechando que las letras ASCII correspondientes se encuentran a una distancia fija. La regla que rige esta operación es que todos los caracteres se convierten a enteros antes de realizarla. Tenga en cuenta que la conversión puede implicar la extensión de signo si el bit más a la izquierda de un carácter es 1; el entero resultante podría ser negativo. (Esto no ocurre con caracteres reales en ninguna máquina actual).

Para convertir un archivo a minúsculas:

```
principal( ) {  
    carácter c;  
    mientras( (c=getchar( )) != '\0' )  
        si( 'A'<=c && c<='Z' )  
            poner carácter(c+'a'-'A');  
    demás  
        putchar(c); }
```

DemásCláusula; Expresiones condicionales

Acabamos de usar undemásdespués de unsi.La forma más general desies

```
si (expresión)declaración1demásdeclaración2
```

el demásEsta parte es opcional, pero a menudo útil. Los conjuntos de ejemplos canónicosincógnitaal mínimo deayb:

```

si (a < b)
    x = a;
demás
    x = b;

```

C proporciona una forma alternativa de condicional que suele ser más concisa. Se denomina "expresión condicional" porque es una condición que realmente tiene un valor y puede utilizarse en cualquier lugar donde se pueda utilizar una expresión. El valor de

```
a < b ? a : b;
```

es asiaes menor que b; es b de lo contrario. En general, la forma

```
expr1 ? expr2 : expr3
```

Para configurarincógnitaal mínimo deayb, entonces:

```
x = (a < b ? a : b);
```

Los paréntesis no son necesarios porque '?' se evalúa antes que '=', pero la seguridad es lo primero.

Dando un paso más allá, podríamos escribir el bucle en el programa en minúsculas como

```

mientras( (c=getchar()) != '\0' )
    putchar( ('A' <= c && c <= 'Z') ? c - 'A' + 'a' : c );

```

Las estructuras if y else se pueden usar para construir lógica que se ramifica en varias direcciones y luego se vuelve a unir, una estructura de programación común, de esta manera:

```

si(...)
    {...}
si no(...)
    {...}
si no(...)
    {...}
demás    {...}

```

Las condiciones se prueban en orden y se ejecuta exactamente un bloque; ya sea el primero cuyo if se cumple o el último demás. La instrucción ejecutada es la que sigue al último else. Caso "predeterminado", omitir el último demás.

no se va a tomar ninguna medida para el

Por ejemplo, para contar letras, dígitos y otros en un archivo, podríamos escribir:

```
principal() {
    int let, dig, other, c; let = dig = other = 0;
    while( (c=getchar()) != '\0' )

        si( ('A'<=c && c<='Z') || ('a'<=c && c<='z') )
            ++dejar;
        else if( '0'<=c && c<='9' ) ++dig; else ++other;

    printf("%d letras, %d dígitos, %d otros\n", let, dig, other);
}
```

Este código contiene letras, dígitos y otros elementos en un archivo.

Operadores de incremento y decremento

Además del habitual '-', C también tiene otros dos operadores unarios interesantes, '++' (incremento) y '--' (decremento). Supongamos que queremos contar las líneas en un archivo.

```
principal() {
    entero c,n;
    n = 0;
    mientras( (c=getchar()) != '\0' )
        si( c == '\n' )
            ++n;
    printf("%d líneas\n", n);
}
```

++ n es equivalente a n=n+1 pero más claro, especialmente cuando n es una expresión compleja. ++ y -- solo se pueden aplicar a enteros y caracteres (y punteros, que aún no hemos visto).

La característica inusual de ++ y -- es que se pueden usar antes o después de una variable. El valor de ++k es el valor de k *después* Se ha incrementado. El valor de k++ es k *antes* se incrementa. Supongamos que k es 5. Entonces

```
x = ++k;
```

incrementa k a 6 y luego establece x al valor resultante, es decir, a 6. Pero

```
x = k++;
```

Primero establece *x* a 5, y *entonces* incrementa *k* a 6. El efecto de incremento de *++k* y *k++* es el mismo, pero sus valores son respectivamente 5 y 6. Pronto veremos ejemplos donde ambos usos son importantes.

Matrices

En C, al igual que en Fortran o PL/I, es posible crear arreglos cuyos elementos sean tipos básicos. Así, podemos crear un arreglo de 10 enteros con la declaración

```
entero x[10];
```

Los corchetes indican subíndices; los paréntesis se utilizan solo para referencias a funciones. Los índices de los arreglos comienzan en cero, por lo que los elementos de *x* son

```
x[0], x[1], x[2], ..., x[9]
```

Si una matriz tiene *n* elementos, el subíndice más grande es *n*-1.

Se proporcionan matrices multidimensionales, aunque no se utilizan mucho más allá de dos dimensiones. La declaración y el uso se ven así:

```
entero nombre[10] [20];  
n = nombre[i+j] [1] + nombre[k] [2];
```

Los subíndices pueden ser expresiones enteras arbitrarias. Las matrices multidimensionales se almacenan por filas (a diferencia de Fortran), por lo que el subíndice situado más a la derecha varía más rápidamente; la matriz `nombre` tiene 10 filas y 20 columnas.

Aquí hay un programa que lee una línea, la almacena en un búfer e imprime su longitud (excluyendo el salto de línea al final).

```
principal() {  
    int n, c;  
    char línea[100];  
    n = 0;  
    while( (c=getchar( )) != '\n' ) {  
        si( n < 100 )  
            línea[n] = c;  
        n++;  
    }  
    printf("longitud = %d\n", n);  
}
```

Como problema más complejo, supongamos que queremos imprimir el recuento de cada línea de la entrada, conservando los primeros 100 caracteres de cada línea. Inténtalo como ejercicio antes de ver la solución:

```
principal( ) {
    int n, c; char line[100]; n = 0;

    mientras( (c=getchar( )) != '\0' )
        si( c == '\n' ) {
            printf("%d", n);
            n = 0;
        }
        demás {
            if( n < 100 ) line[n] = c; n++;
        }
}
```

El código anterior almacena los primeros 100 caracteres de cada línea.

Matrices de caracteres; Cadenas de caracteres

El texto se suele almacenar como una matriz de caracteres, como hicimos con `line[ ]` en el ejemplo anterior. Por convención en C, el último carácter de una matriz de caracteres debe ser `'\0'` porque la mayoría de los programas que manipulan matrices de caracteres lo esperan. Por ejemplo, `printf` usa `'\0'` para detectar el final de una matriz de caracteres al imprimirla con `'%s'`.

Podemos copiar una matriz de caracteres `s` en otra `t` de esta manera:

```
i = 0;
mientras( (t[i]=s[i]) != '\0' )
    i++;
```

La mayoría de las veces tenemos que insertar nuestro propio `'\0'` al final de una cadena; si queremos imprimir la línea con `printf`, es necesario. Este código imprime el número de caracteres antes de la línea:

```
principal( ) {
    entero n;
    char línea[100];
    n = 0;
    while( (linea[n++]=getchar( )) != '\n' ); linea[n] = '\0';

    printf("%d:\t%s", n, línea);
}
```

Aquí incrementamos *n* en el propio subíndice, pero solo después de que se haya utilizado el valor anterior. Se lee el carácter, se coloca en *line[n]* y solo entonces se incrementa *n*.

Hay un único lugar donde C inserta automáticamente el carácter `'\0'` al final de una matriz de caracteres, y ese lugar es en la construcción.

"cosas entre comillas dobles"

El compilador agrega automáticamente un `'\0'` al final. El texto encerrado entre comillas dobles se llama *cadena*; sus propiedades son precisamente las de una matriz de caracteres (inicializada).

paraDeclaración

La instrucción `for` es una versión algo generalizada del bucle `while` que nos permite colocar las partes de inicialización e incremento de un bucle en una sola instrucción junto con la prueba. La forma general del bucle `for` es:

```
para(inicialización; expresión; incremento)  
    declaración
```

El significado es exactamente

```
inicialización;  
mientras(expresión) {  
    declaración  
    incremento;  
}
```

Este ejemplo, un poco más elaborado, suma los elementos de una matriz:

```
suma = 0;  
para (i = 0; i < n; i++) suma = suma + array[i];
```

En la instrucción `for`, la inicialización se puede omitir si se desea, pero el punto y coma debe estar presente. El incremento también es opcional. Es *no* seguido de un punto y coma. La segunda cláusula, la prueba, funciona de la misma manera que en el `mientras`: si la expresión es verdadera (no cero), realiza otro bucle; de lo contrario, continúa con la siguiente instrucción. Al igual que con el `mientras`, el bucle `for` puede ejecutarse cero veces. Si se omite la expresión, se considera que siempre es verdadera, por lo que

```
para( ; ) ...  
mientras( 1 ) ...
```

ambos son bucles infinitos.

Quizás te preguntes por qué usamos un `para` que se parece mucho a un `mientras`. (También podrías preguntar por qué usamos un `mientras` porque...) El `para` Por lo general, es preferible porque mantiene el código donde se usa `y`, a veces, elimina la necesidad de sentencias compuestas, como en este código que pone a cero una matriz bidimensional:

```
para( i=0; i<n; i++ )
    para( j=0; j<m; j++ )
        array[i][j] = 0;
```

Funciones; Comentarios

Supongamos que queremos, como parte de un programa más amplio, contar las ocurrencias de caracteres ASCII en un texto de entrada. Agrupemos también los caracteres no válidos (aquellos con valor > 127 o < 0) en una sola pila. Dado que presumiblemente se trata de una parte aislada del programa, la buena práctica recomienda que sea una función independiente. He aquí una forma de hacerlo:

```
principal( ) {
    int hist[129];          /* 128 caracteres legales + 1 grupo ilegal */
    ...
    contar(hist, 128);      /* cuenta las letras en hist */ /* los comentarios
    printf( ... );          tienen este aspecto; úsalos */
    ...
}

Contar(buf, tamaño)
    tamaño entero, buf[ ]; {
    entero i, c;
    para( i=0; i<=tamaño; i++ )
        buf[i] = 0;          /* establecer búfer a cero */ /*
    mientras( (c=getchar( )) != '\0' ) { if( c > size || c      leer hasta el final del archivo */
        < 0 )
        c = tamaño;          /* corregir entrada no válida */
        buf[c]++;
    }
    devolver;
}
```

Ya hemos visto muchos ejemplos de cómo llamar a una función, así que centrémonos en cómo definirla. Como ``count`` tiene dos argumentos, debemos declararlos, como se muestra, indicando sus tipos `y`, en el caso de ``buf``, que se trata de un array. Las declaraciones de los argumentos se colocan entre la lista de argumentos y la llave de apertura ``{``. No es necesario especificar el tamaño del array ``buf``, ya que se define fuera de ``count``.

La instrucción `return` simplemente indica que se vuelva a la rutina que la llamó. De hecho, podríamos haberla omitido, ya que el retorno está implícito al final de una función.

¿Qué ocurre si queremos que la función `count` devuelva un valor, por ejemplo, el número de caracteres leídos? La instrucción `return` también permite esto:

```
entero i, c, nchar;
nchar = 0;
...
while( (c=getchar( )) != '\0' ) {
    si( c > tamaño || c < 0 )
        c = tamaño;
    buf[c]++;
    nchar++;
}
devolver(nchar);
```

Cualquier expresión puede aparecer dentro de los paréntesis. Aquí hay una función para calcular el mínimo de dos números enteros:

```
mínimo(a, b)
    entero a, b; {
        return( a < b ? a : b );
    }
```

Para copiar una matriz de caracteres, podríamos escribir la función

```
strcpy(s1, s2) /* copia s1 a s2 */
    char s1[ ], s2[ ]; {
        entero i;
        para( i = 0; (s2[i] = s1[i]) != '\0'; i++ );
    }
```

Como suele ocurrir, todo el trabajo lo realiza la instrucción de asignación integrada en la parte de prueba del bucle `for`. De nuevo, las declaraciones de los argumentos `s1` y `s2` omiten los tamaños, ya que no son relevantes para `strcpy`. (En la sección sobre punteros, veremos una forma más eficiente de copiar cadenas).

Existe una sutileza en el uso de funciones que puede engañar al programador de Fortran desprevenido. En C, las variables simples (no los arreglos) se pasan mediante `call by value`, lo que significa que la función llamada recibe una copia de sus argumentos y desconoce sus direcciones. Esto imposibilita modificar el valor de uno de los argumentos de entrada.

Hay dos maneras de salir de este dilema. Una es hacer arreglos especiales para pasar a la función la dirección de una variable en lugar de su valor. La otra es hacer que la variable

una variable global o externa, conocida por cada función por su nombre. Analizaremos ambas posibilidades en las próximas secciones.

Variables locales y externas

Si decimos

```
f() {  
    entero x;  
    ...  
}  
g() {  
    entero x;  
    ...  
}
```

cada *x* es *local* a su propia rutina: la *x* en *f* no está relacionada con la *x* en *g*. (Las variables locales también se denominan "automáticas"). Además, cada variable local en una rutina aparece solo cuando se llama a la función, y *desaparece* Cuando se sale de la función, las variables locales no conservan memoria entre llamadas y deben inicializarse explícitamente en cada entrada. (Existe una clase de almacenamiento estático para crear variables locales con memoria; no la analizaremos).

A diferencia de las variables locales, las variables externas se definen fuera de todas las funciones y están (potencialmente) disponibles para todas ellas. El almacenamiento externo siempre existe. Para que las variables sean externas, debemos definirlas fuera de todas las funciones y, donde queramos utilizarlas, declararlas.

```
principal() {  
    extern int nchar, hist[ ]; ...  
  
    contar( );  
    ...  
}  
contar() {  
    extern int nchar, hist[ ]; int i, c;  
  
    ...  
}  
  
entero    hist[129];          /* espacio para el histograma */  
entero    nchar;              * recuento de caracteres */
```

En términos generales, cualquier función que desee acceder a una variable externa debe incluir una declaración ``extern`` para dicha variable. La declaración es la misma que las demás, salvo por la adición de la palabra clave ``extern``. Además, debe existir una definición de las variables externas a todas las funciones.

Las variables externas pueden inicializarse; se establecen a cero si no se inicializan explícitamente. En su forma más simple, la inicialización se realiza colocando el valor (que debe ser una constante) después de la definición:

Consejos

Un puntero en C es la dirección de algo. Es un caso realmente raro cuando nos importa cuál es la dirección específica. La dirección en sí misma lo es, pero los punteros son una forma bastante común de acceder al contenido de algo. El operador unario `&` se utiliza para producir la dirección de un objeto, si la tiene. Por lo tanto

```
entero a, b;
b = &a;
```

pone la dirección de `a` en `b`. No podemos hacer mucho con él excepto imprimirlo o pasarlo a alguna otra rutina, porque no lo hemos dado una declaración correcta. Pero si declaramos que `b` es efectivamente un puntero a un número entero, estamos en buena forma:

```
entero a, *b, c;
b = &a;
c = *b;
```

`b` contiene la dirección de `a` y `*c = *b` significa usar el valor en `b` como una dirección, es decir, como un puntero. El efecto es que recuperamos el contenido de `a`, aunque de forma bastante indirecta. (Siempre es el caso que `*&x` es lo mismo que `x` si `x` tiene una dirección.)

El uso más frecuente de punteros en C es para recorrer eficientemente los arreglos. De hecho, en la implementación de un arreglo, el nombre del arreglo representa la dirección del elemento cero del arreglo, por lo que no se puede usar en el lado izquierdo de una expresión. (No se puede cambiar la dirección de algo asignándole un valor). Si decimos

```
char *y; char x[100];
```

`y` es de tipo puntero a carácter (aunque todavía no apunta a ningún sitio). Podemos hacer y apuntar a un elemento de `x` por cualquiera de

```
y = &x[0];
y = x;
```

Desde `x` la dirección de `x[0]` es legal y coherente.

Ahora `*y` da `x[0]`. Más importante aún,

```
* (y+1) da x[1]
* (y+i) da x[i]
```


y la secuencia

```
y = &x[0];  
y++;
```

hojasyapuntando a x[1].

Usemos punteros en una función llamada `length` que calcule la longitud de un array de caracteres. Recuerda que, por convención, todos los arrays de caracteres terminan con un `'\0'`. (Y si no es así, este programa fallará inevitablemente). La forma antigua:

```
longitud(es)  
char s[ ]; {  
    entero n;  
    para( n=0; s[n] != '\0'; )  
        n++;  
    devolver(n);  
}
```

Reescribir con punteros da como resultado

```
longitud(es)  
carácter *s; {  
    entero n;  
    para( n=0; *s != '\0'; s++ )  
        n++;  
    devolver(n); }
```

Ahora puedes ver por qué tenemos que decir qué tipo de cosa apunta a -- si vamos a incrementarlo con `s++`. Tenemos que incrementarlo en la cantidad correcta.

La versión con puntero es más eficiente (esto es casi siempre cierto) pero aún más compacta es

```
para( n=0; *s++ != '\0'; n++ );
```

El `\*s` devuelve un carácter; el `++` incrementa el puntero para que obtengamos el siguiente carácter. La próxima vez. Como puedes ver, a medida que hacemos las cosas más eficientes, también las hacemos menos claras. Pero `\*s++` es una expresión tan común que tienes que conocerla.

Dando un paso más allá, aquí está nuestra función `strcpy` que copia una matriz de caracteres s a otra t.

```
strcpy(s,t)  
char *s, *t; {  
    mientras(*t++ = *s++);  
}
```

Hemos omitido la prueba contra '\0', porque '\0' es idénticamente cero; a menudo verá el código de esta manera.

Para los argumentos de una función, y solo allí, las declaraciones

```
carácter s[ ];  
carácter *s;
```

son equivalentes: un puntero a un tipo, o una matriz de tamaño no especificado de ese tipo, son lo mismo.

Argumentos de función

Mira hacia atrás a [la función strcpy en la sección anterior](#). Le pasamos dos nombres de cadena como argumentos, luego procedimos a sobrescribirlos ambos mediante incremento. Entonces, ¿cómo es que no perdemos las cadenas originales en la función que llamó a strcpy?

Como dijimos antes, C es un lenguaje de "paso por valor": cuando haces una llamada a una función como `f(x)`, *valor* de `incógnita` se pasa, no su dirección. Por lo tanto, no hay manera *de alterar* `incógnita` desde dentro. Si `incógnita` es un array (`char x[10]`) esto no es un problema, porque `incógnita` es una dirección de todos modos, y no estás intentando cambiarla, solo a qué apunta. Por eso `strcpy` funciona como debe. Y es conveniente no tener que preocuparse por hacer copias temporales de los argumentos de entrada.

Pero ¿y si...? `incógnita` ¿Es un escalar y quieres cambiarlo? En ese caso, tienes que pasar el *DIRECCIÓN* de `incógnita` a `f` y luego usarlo como puntero. Así, por ejemplo, para intercambiar dos enteros, debemos escribir

```
voltear(x, y)  
    entero *x, *y; {  
        temperatura entera;  
        temp = *x;  
        * x = *y;  
        * y = temperatura;  
    }
```

y llamar `voltear`. Tenemos que pasar las direcciones de las variables:

```
voltear (&a, &b);
```

Que intercambian dos números enteros.

La instrucción Switch; Break; Continue

La instrucción switch se puede utilizar para reemplazar la prueba multi-vía que utilizamos en el último Ejemplo. Cuando las pruebas son como esta:

```
si( c == 'a' ) ... si no si( c ==  
'b' ) ... si no si( c == 'c' ) ... si  
no ...
```

Para comparar un valor con una serie de constantes, la instrucción switch suele ser más clara y generalmente produce un código mejor. Úsela de esta manera:

```
interruptor( c ) {  
    caso 'a':  
        bandera++;  
        romper;  
    caso 'b':  
        bflag++;  
        romper;  
    caso 'c':  
        cflag++;  
        romper;  
    por defecto:  
        printf("%c?\n", c);  
        romper;  
}
```

Las declaraciones de casos identifican las distintas acciones que deseamos realizar; por defecto se realiza si no se satisface ninguno de los otros casos. (A por defecto (Es opcional; si no está ahí y ninguno de los casos coincide, simplemente te caes por el fondo).

La instrucción `break` en este ejemplo es nueva. Está ahí porque los casos son solo etiquetas, y después de ejecutar uno de ellos, se pasa al siguiente a menos que se realice alguna acción explícita para salir. Esto tiene sus ventajas y desventajas. Por un lado, se pueden tener varios casos en una sola instrucción; podríamos querer permitir tanto mayúsculas como minúsculas.

Pero ¿qué pasa si simplemente queremos salir después de hacer el caso 'a'? Podríamos salir de un caso del cambiar con una etiqueta y unair a, pero esto es realmente feo. La instrucción break nos permite salir sin ninguno de los dos ir a o etiqueta.

La instrucción break también funciona en las instrucciones for y while; provoca una salida inmediata del bucle.

La instrucción `continue` dentro de un bucle `while`; provoca que se inicie la siguiente iteración del bucle. Esto significa que va a la parte de incremento del parámetro y la parte de prueba de la `while`.

Estructuras

El uso principal de las estructuras es agrupar colecciones de tipos de variables dispares, por lo que Pueden tratarse convenientemente como una unidad. Por ejemplo, si estuviéramos escribiendo un compilador o un ensamblador, podríamos necesitar para cada identificador información como su nombre (una matriz de caracteres), su número de línea de código fuente (un entero), alguna información de tipo (un carácter, tal vez) y probablemente un contador de uso (otro entero).

```

carbonizarse    id[10];
entero         línea;
carbonizarse    tipo;
entero         uso;

```

Podemos crear una estructura a partir de esto con bastante facilidad. Primero le indicamos a C cómo será la estructura, es decir, qué tipo de elementos contendrá; después, podemos reservar espacio de almacenamiento para ella, ya sea en la misma instrucción o por separado. Lo más sencillo es definirla y asignar el espacio de almacenamiento de una sola vez:

```

struct {
    carbonizarse    id[10];
    entero         línea;
    carbonizarse    tipo;
    entero         uso;
} sim;

```

Esto define a `sym` como una estructura con la forma especificada; `identificación`, `línea`, `tipo` y `uso` son miembros de la estructura. La forma en que nos referimos a cualquier miembro particular de la estructura es

```
nombre-de-estructura . miembro
```

como en

```

tipo de símbolo = 077;
si( sym.usage == 0 ) ...
mientras( sym.id[j++] ) ...
etc.

```

Aunque los nombres de los miembros de una estructura nunca aparecen solos, deben ser únicos; no puede haber otro identificador ni utilizarse en otra estructura.

Hasta ahora no hemos ganado mucho. Las ventajas de las estructuras comienzan a notarse cuando tenemos matrices de estructuras, o cuando queremos pasar diseños de datos complejos entre funciones. Supongamos que quisiéramos crear una tabla de símbolos para hasta 100 identificadores. Podríamos extender nuestras definiciones como

```
carbonizarse id[100][10];
entero línea[100];
carbonizarse tipo[100];
entero uso[100];
```

pero una estructura nos permite reorganizar esta información dispersa de manera que todos los datos sobre un único identificador se recopilen en un solo bloque:

```
struct {
    carbonizarse id[10];
    entero línea;
    carbonizarse tipo;
    entero uso;
} sym[100];
```

Esto convierte a sym en una matriz de estructuras; cada elemento de la matriz tiene la forma especificada. Ahora tenemos pueden referirse a los miembros como

```
sym[i].usage++; /* incrementa el uso del i-ésimo identificador */
para( j=0; sym[i].id[j++] != '\0'; ) ...
    etc.
```

Por lo tanto, para imprimir una lista de todos los identificadores que no se han utilizado, junto con su número de línea,

```
para( i=0; i<nsym; i++ )
    si( sym[i].usage == 0 )
        printf("%d\t%s\n", sym[i].line, sym[i].id);
```

Supongamos que ahora queremos escribir una función lookup(name) que nos diga si name ya existe en sym, indicando su índice, o no indicando que no lo tiene, devolviendo -1. No podemos pasar una estructura directamente a una función; debemos definirla externamente o pasarle un puntero. Probemos primero con la primera opción.

```
entero nsym 0; /* longitud actual de la tabla de símbolos */

struct {
    carbonizarse id[10];
    entero línea;
    carbonizarse tipo;
    entero uso;
} sym[100]; /* tabla de símbolos */
```

```

principal() {
    ...
    si ( (índice = búsqueda(nuevonombre)) >= 0 )
        sym[index].usage++;          /* ya está ahí ... */
    demás
        instalar(nuevonombre, nuevalínea, nuevotipo);
    ...
}

búsqueda(s)
carácter *s; {
    entero i;
    extern struct {
        carbonizarse id[10];
        entero línea;
        carbonizarse tipo;
        entero uso;
    } sym[ ];

    para( i=0; i<nsym; i++ )
        si( compar(s, sym[i].id) > 0 )
            devolver(i);

    devolver(-1);
}

comparar(s1,s2)          /* Devuelve 1 si s1==s2, 0 en caso contrario */
carácter *s1, *s2; {
    mientras( *s1++ == *s2 )
        si( *s2++ == '\0' )
            devolver(1);

    devolver(0);
}

```

No es necesario declarar la estructura en la búsqueda si la definición externa precede a su uso en el mismo archivo fuente, como veremos en un momento.

¿Y si queremos usar punteros?

```

estructura symtag {
    carbonizarse id[10];
    entero línea;
    carbonizarse tipo;
    entero uso;
} sym[100], *psym;

psym = &sym[0]; /* o p = sym; */

```

Esto hace que `psym` un puntero a nuestro tipo de estructura (la tabla de símbolos), luego lo inicializa para que apunte al primer elemento `desim`.

Observe que agregamos algo después de la palabra struct: una "etiqueta" llamada symtag. Esto le da un nombre a la definición de nuestra estructura para que podamos referirnos a ella más adelante sin repetir la definición. No es necesario, pero sí útil. De hecho, podríamos haber dicho:

```
estructura    symtag {
               ... definición de estructura
};
```

lo cual no habría asignado ningún almacenamiento en absoluto, y luego dijo

```
estructura    etiqueta de símbolo    sim[100];
estructura    etiqueta de símbolo    * psym;
```

lo que definiría el array y el puntero. Esto podría condensarse aún más, a

```
struct symtag sym[100], *psym;
```

La forma en que nos referimos a un miembro de una estructura mediante un puntero es la siguiente:

```
puntero -> miembro de la estructura
```

El símbolo `->' significa que estamos señalando a un miembro de una estructura; `->' solo se usa en ese contexto. puntero es un puntero a la (base de) una estructura que contiene el miembro de la estructura. La expresión ptr->structure-member se refiere al miembro indicado de la estructura a la que apunta. Tenemos construcciones como:

```
psym->type = 1;
psym->id[0] = 'a';
```

Para expresiones de puntero más complejas, es recomendable usar paréntesis para dejar claro qué va con qué. Por ejemplo,

```
struct { int x, *y; } *p; p->x++
           incrementos x
+ + p->x   ¡Esto también!
(++p)->x   incrementa p antes de obtener x
* p->y++ usa y como puntero y luego lo incrementa.
* (p->y)++ Esto también lo hace
* (p++)->y utiliza y como puntero, luego incrementa p
```

La forma de recordarlos es que -, . (punto), () y [] se enlazan muy estrechamente. Una expresión Si uno de estos elementos se trata como una unidad. p->x, a[i], yxypensión completa)son nombres exactamente como abecedarios.

Si pages un puntero a una estructura, cualquier aritmética en pag tiene en cuenta el tamaño real de la estructura. Por ejemplo, p++ incrementos pag por la cantidad correcta para obtener el siguiente elemento del arreglo de estructuras. Pero no asuma que el tamaño de una estructura es la suma de los tamaños.

de sus miembros: debido a la alineación de objetos de diferentes tamaños, puede haber "agujeros" en una estructura.

Basta de teoría. Aquí tenéis un ejemplo de búsqueda, esta vez con punteros.

```

struct symtag {
    carbonizarse id[10];
    entero línea;
    carbonizarse tipo;
    entero uso;
} sym[100];

principal( ) {
    struct symtag *lookup(); struct
    symtag *psym;
    ...
    if( (psym = lookup(newname)) ) psym -> /* puntero distinto de cero */ /*
    usage++;                          significa que ya está ahí */
    demás
        instalar(nuevonombre, nuevalínea, nuevotipo);
    ...
}

struct symtag *lookup(s)
    carácter *s; {
    struct symtag *p;
    para( p=sym; p < &sym[nsym]; p++ )
        si( compar(s, p->id) > 0)
            devolver(p);
    devolver(0);
}

```

La función compar no cambia: `p->id' se refiere a una cadena.

En principal probamos el puntero devuelto por buscar frente a cero, basándose en el hecho de que un puntero, por definición, nunca es cero cuando realmente apunta a algo. Las demás manipulaciones de punteros son triviales.

La única complejidad es el conjunto de líneas como

```
struct symtag *lookup();
```

Esto nos lleva a un tema que trataremos brevemente: la cuestión de los tipos de funciones. Hasta ahora, todas nuestras funciones han devuelto enteros (o caracteres, que son prácticamente lo mismo). ¿Qué hacemos cuando la función devuelve algo más, como un puntero a una estructura? La regla es que cualquier función que no devuelva un entero debe indicar explícitamente qué devuelve. La información de tipo va antes del nombre de la función (lo que puede dificultar la lectura del nombre).

Ejemplos:

```

carácter f(a)
    entero a; {
        ...
    }
int *g( ) { ... }

struct symtag *lookup(s) char *s; { ... }

```

La función `f` devuelve un personaje, `g` devuelve un puntero a un entero, y `lookup` devuelve un puntero a una estructura que se parece a `symtag`. Y si vamos a usar una de estas funciones, tenemos que hacer una declaración donde la usamos, como hicimos en principal arriba.

Observe el paralelismo entre las declaraciones.

```

struct symtag *lookup(); struct
symtag *psym;

```

En efecto, esto significa que `lookup` y `psym` Ambos se utilizan de la misma manera, como un puntero a una estructura, aunque uno sea una variable y el otro una función.

Iniciación de variables

Una variable externa puede inicializarse en tiempo de compilación añadiendo un guion después de su nombre. Valor inicializado cuando se define. El valor inicializado debe ser algo cuyo valor se conozca en tiempo de compilación, como una constante.

```

entero    x = 0; /* "0" podría ser cualquier constante */ /*
entero    * pag &y[1]; p ahora apunta a y[1] */

```

Se puede inicializar un array externo añadiendo después de su nombre una lista de inicializaciones encerradas entre llaves:

```

entero    x[4] = {0,1,2,3}; /* hace que x[i] = i */
entero    y[ ] = {0,1,2,3}; /* hace que y sea lo suficientemente grande para 4 valores */
carbonizarse * msg = "error de sintaxis\n"; /* las llaves no son necesarias aquí */
char *palabra clave[ ]={
    "si",
    "demás",
    "para",
    "mientras",
    "romper",
    "continuar",
    0
};

```

Esta última es muy útil: convierte la palabra clave en una matriz de punteros a cadenas de caracteres, con un cero al final para que podamos identificar fácilmente el último elemento. Una rutina de búsqueda simple.

podría escanear esto hasta que encuentre una coincidencia o encuentre un puntero de palabra clave cero:

```

buscar(str)                /* buscar str en keyword[ ] */
    carácter *str; {
        entero i,j,r;
        para( i=0; palabra_clave[i] != 0; i++) {
            para( j=0; (r=palabra clave[i][j]) == str[j] && r != '\0'; j++)
                si( r == str[j] )
                    devolver(i);
        }
        devolver(-1);
    }

```

Reglas de alcance

Un programa C completo no necesita compilarse de una sola vez; el texto fuente del programa puede mantenerse en varios archivos, y las rutinas previamente compiladas pueden cargarse desde Bibliotecas. ¿Cómo organizamos el paso de datos entre rutinas? Ya hemos visto cómo usar argumentos y valores de funciones, así que hablemos de datos externos. Advertencia: en esta sección se utilizan los términos «declaración» y «definición» con precisión; no los confunda.

Existe un atajo importante para hacer declaraciones extern. Si la definición de una variable aparece *antes* su uso en alguna función, no se necesita ninguna declaración extern dentro de la función. Por lo tanto, si un archivo contiene

```

f1() { ... }
entero foo;
f2() { ... foo = 1; ... } f3() { ... si
( foo ) ... }

```

ninguna declaración de `foo` es necesario en cualquiera de los dos `f2` o `f3`, porque la definición externa de `foo` aparece ante ellos. Pero si `f1` quiere usar `foo`, debe contener la declaración

```

f1() {
    extern int foo;
    ...
}

```

Esto también es cierto para cualquier función que exista en otro archivo; si lo desea, puede tener que usar una Declaración externa para ello. (Si en algún lugar hay una declaración externa para algo, también debe haber eventualmente una definición externa para ello, o recibirá un mensaje de "símbolo indefinido").